

Take-Home Assignment: Sportsbook RAG Assistant

(AI Solutions Engineer – Jan. 2026)

Scope

You're building an internal assistant for an online sports betting company. It should answer ops questions using retrieval-augmented generation (RAG) over a small table of bet records.

You will be given a small dataset (note: some names are intentionally mis-spelled which don't have any effect on the output – you do not need to do any actions on them):

- bets_100_10cols.csv (100 rows)

Your assistant must retrieve relevant rows and then use an LLM to generate a grounded answer with citations (bet IDs).

Dataset: bets_100_10cols.csv (10 columns)

Each row is one bet, and the columns are:

Column	Type	Description
bet_id	string	Unique identifier for the bet (e.g., B0042). Use this for citations.
customer_id	string	Identifier for the customer who placed the bet (e.g., C029). Not PII.
sport	string	One of: football, tennis, basketball.
event_name	string	Fixture/match name. Contains intentionally slightly misspelled team/player names (e.g., Bulls vs Rapters). Treat as noisy text.
market	string	Bet type/market (e.g., Match Winner, Moneyline, Over/Under 2.5, Set 1 Winner).
selection	String	Selected outcome. For team/player markets it matches event_name spelling; for totals/props it can be values like Over 215.5, Under 21.5, Yes/No, etc.

stake_gbp	Number	Stake amount in GBP.
status	string	Lifecycle state: SETTLED, PENDING, VOID, REJECTED.
incident_tag	string	Operational tag: NONE, LATENCY_SPIKE, FEED_OUTAGE, MARKET_SUSPENDED, MANUAL_REVIEW.
price_delay_ms	integer	Approx. latency (ms) between feed update and offered price at the time (synthetic). Higher values imply staler pricing / degraded experience.

Your tasks

1) Build ingestion + retrieval

- Load the CSV and/or build a local database.
- Decide how to represent each row as text (e.g., concatenate fields into a "bet document").
- Ensure retrieval can handle queries by bet_id, customer-level queries by customer_id, incident/latency queries by incident_tag/price_delay_ms, and event names.

2) Build the RAG (with citations)

- Implement a CLI (or if you wish, a small web endpoint so that the interviewers can run on their ends) that takes a user question and returns:
 - A short, helpful answer grounded in retrieved rows.
 - Citations as bet_id values used (e.g., "Evidence: B0021, B0044").
 - If evidence is insufficient, say so and ask for the missing key (e.g., bet_id/customer_id).

Example prompts your assistant should handle:

- Why was bet BXXYZ rejected?
- Which customers were most impacted by LATENCY_SPIKE?
- Find the highest price_delay_ms bets and summarize what happened.
- For customer CXYZ, summarize their last few bets and any incidents.

3) Provide 6 acceptance questions (e.g., above bullet points) + outputs

- Include 6 test questions (you choose) and paste the outputs from your system.
- Please include at least 2 bet-specific questions (single bet_id), 2 analytics-style questions (aggregations/ranking), and 2 incident/ops questions (latency/suspensions).

Deliverables

- Source code (repo zip or link).
- README with how to run, architecture overview (e.g., ingestion, retrieval, prompt, response), tradeoffs and what you'd improve with more time.
- NOTE: It's OK to use AI assistants such as ChatGPT but please be prepared to understand your code as well as for some follow-up questions.